

Lab 3 — Colisión de MD5

Vulnerabilidad: una función hash criptográfica debe ser *resistente a colisiones*: debe ser inviable encontrar dos entradas distintas con el mismo hash. MD5 se rompió en 2004 (Wang et al.) y hoy se generan colisiones en segundos. SHA-1 cayó en 2017 (ataque *SHAttered* de Google). Un hash sin resistencia a colisiones no sirve para integridad ni para firmas.

El reto

Los bloques A y B son dos archivos distintos (difieren en 6 bytes). Demuestra que MD5 no distingue archivos y explica qué ataque habilita.

LABORATORIO · BÁSICO

3 · Colisión de MD5

MD5 (Y SHA-1) YA NO RESISTEN COLISIONES

🎯 OBJETIVO

Demstrar que un hash roto no sirve para garantizar integridad.

EL RETO

Los bloques A y B son dos archivos DISTINTOS (difieren en 6 bytes). El flujo ya calcula el MD5 de cada uno: son idénticos. Ahora agrega a cada rama un bloque «SHA2» (256) y compara. Responde: si un antivirus o una tienda de apps identifica los archivos por su MD5, ¿qué ataque permite esto?

💡 PISTAS (3) ▾

LO LOGRASTE CUANDO...

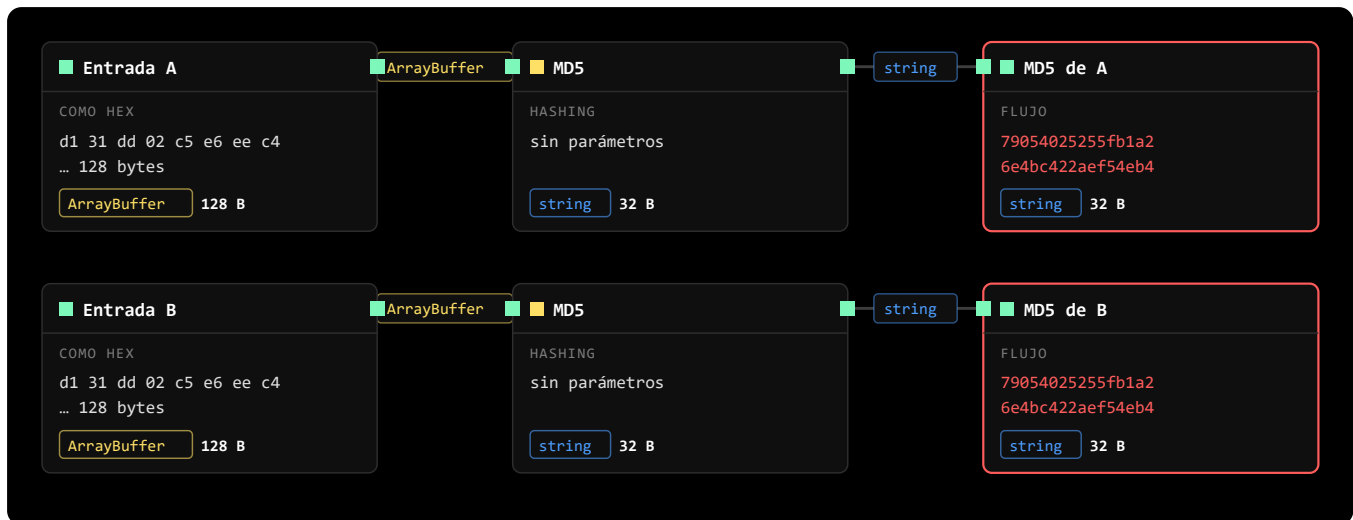
Los dos MD5 coinciden y los dos SHA-256 difieren.

🔗 VER SOLUCIÓN (WRITEUP)

AL LIENZO →

Solución

El flujo calcula `MD5(A)` y `MD5(B)`:



Idénticos, aunque $A \neq B$. Este es el par clásico de Wang et al.: dos bloques que difieren en las posiciones 19, 45, 59, 83, 109 y 123. Puedes verlo abriendo cada *Entrada* y comparando su volcado hexadecimal:

Bloque A y bloque B: 128 bytes cada uno, distintos solo en 6 posiciones
 Posiciones 19, 45, 59, 83, 109 y 123 (resaltadas). Todo lo demás es idéntico.

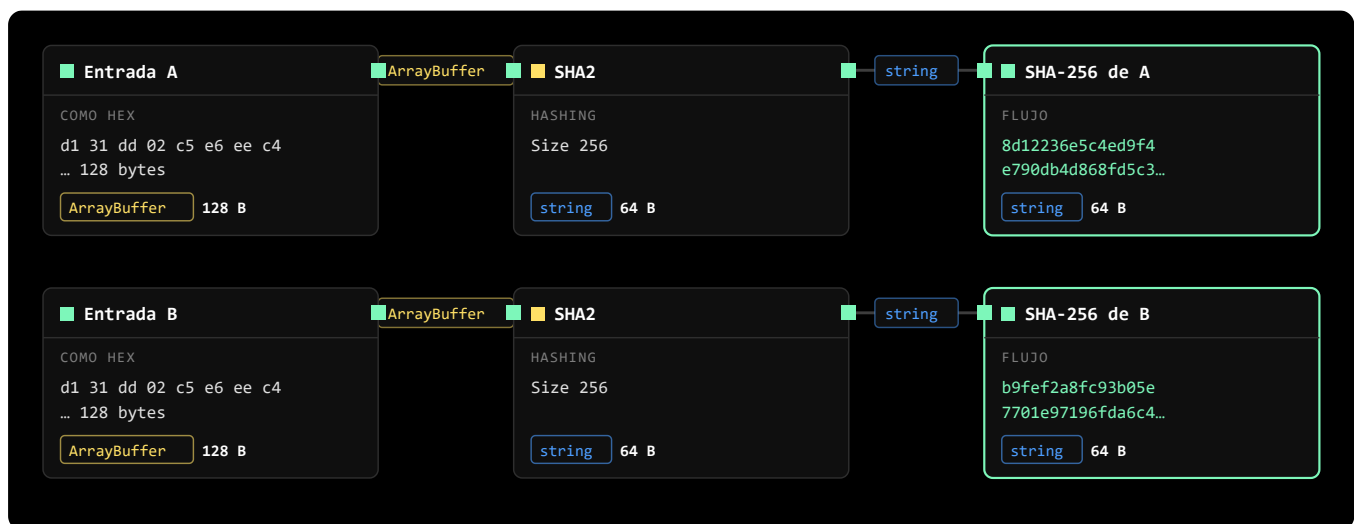
A	B
00 d1 31 dd 02 c5 e6 ee c4 69 3d 9a 06 98 af f9 5c	00 d1 31 dd 02 c5 e6 ee c4 69 3d 9a 06 98 af f9 5c
10 2f ca b5 87 12 46 7e ab 40 04 58 3e b8 fb 7f 89	10 2f ca b5 07 12 46 7e ab 40 04 58 3e b8 fb 7f 89
20 55 ad 34 06 09 f4 b3 02 83 e4 88 83 25 71 41 5a	20 55 ad 34 06 09 f4 b3 02 83 e4 88 83 25 f1 41 5a
30 08 51 25 e8 f7 cd c9 9f d9 1d bd f2 80 37 3c 5b	30 08 51 25 e8 f7 cd c9 9f d9 1d bd 72 80 37 3c 5b
40 d8 82 3e 31 56 34 8f 5b ae 6d ac d4 36 c9 19 c6	40 d8 82 3e 31 56 34 8f 5b ae 6d ac d4 36 c9 19 c6
50 dd 53 e2 b4 87 da 03 fd 02 39 63 06 d2 48 cd a0	50 dd 53 e2 34 87 da 03 fd 02 39 63 06 d2 48 cd a0
60 e9 9f 33 42 0f 57 7e e8 ce 54 b6 70 80 a8 0d 1e	60 e9 9f 33 42 0f 57 7e e8 ce 54 b6 70 80 28 0d 1e
70 c6 98 21 bc b6 a8 83 93 96 f9 65 2b 6f f7 2a 70	70 c6 98 21 bc b6 a8 83 93 96 f9 65 ab 6f f7 2a 70

MD5	A: 79054025255fb1a26e4bc422aef54eb4	B: 79054025255fb1a26e4bc422aef54eb4	iguales
SHA-256	A: 8d12236e5c4ed9f4e790db4d...	B: b9fef2a8fc93b05e7701e971...	distintos

El paso del reto: agrega a cada rama un bloque **SHA2** (Size 256, el valor por defecto) conectado a su *Entrada*, y una *Salida* para verlo. El lienzo queda así:



Aislando solo las ramas nuevas:



SHA-256 sí los distingue: dos hashes completamente diferentes. La conclusión es directa: si un sistema identifica archivos por su MD5, dos archivos distintos le parecen el mismo.

Por qué importan las 6 diferencias

MD5 procesa el mensaje en bloques de 64 bytes; A y B ocupan dos bloques cada uno. La colisión no es casual: los seis bytes que cambian están calculados para que las diferencias se cancelen dentro de la función de compresión de MD5 (rutas diferenciales): el primer bloque introduce una diferencia controlada en el estado interno y el segundo la anula. Al terminar, el estado es idéntico y el hash también. Como MD5 encadena bloques, además se puede añadir cualquier sufijo común a A y B y seguirán colisionando.

El ataque de Wang produce pares como este; el de **prefijo elegido** (*chosen-prefix collision*, Stevens 2007) va más allá: parte de dos prefijos arbitrarios distintos y calcula bloques que hacen colisionar ambos. Eso es lo que permitió falsificar un certificado de CA real.

Verifícalo con otras herramientas

Crea los dos archivos a partir del hex de las *Entradas* (ver [requisitos](#)):

```
echo d131dd02c5e6eec4693d9a0698aff95c2fcbab58712467eab4004583eb8fb7f8955ad340609f4b30283e4888325
71415a085125e8f7cdc99fd91dbdf280373c5bd8823e3156348f5bae6dacd436c919c6dd53e2b487da03fd02396306d
248cda0e99f33420f577ee8ce54b67080a80d1ec69821bcb6a8839396f9652b6ff72a70 | xxd -r -p > a.bin
echo d131dd02c5e6eec4693d9a0698aff95c2fcbab50712467eab4004583eb8fb7f8955ad340609f4b30283e4888325
f1415a085125e8f7cdc99fd91dbdf280373c5bd8823e3156348f5bae6dacd436c919c6dd53e23487da03fd02396306d
248cda0e99f33420f577ee8ce54b67080280d1ec69821bcb6a8839396f965ab6ff72a70 | xxd -r -p > b.bin
```

Compáralos byte a byte:

```
cmp -l a.bin b.bin
# 20 207 7
# 46 161 361
# 60 362 162
# 84 264 64
# 110 250 50
# 124 53 253
```

`cmp -l` cuenta las posiciones desde 1 y muestra los valores en **octal**: `20 207 7` es la posición 19 (contando desde 0, como el volcado de CipherFlow), con `0x87` en A y `0x07` en B. Son las 6 diferencias del diagrama.

Los hashes, con `coreutils` o con `openssl dgst`:

```
md5sum a.bin b.bin
# 79054025255fb1a26e4bc422aef54eb4 a.bin
# 79054025255fb1a26e4bc422aef54eb4 b.bin ← iguales
sha256sum a.bin b.bin
# 8d12236e5c4ed9f4e790db4d868fd5c399df267e18ff65c1107c328228cffc98 a.bin
# b9fef2a8fc93b05e7701e97196fda6c4fbee25ff8e64fdfee7015eca8fa617d b.bin
openssl dgst -sha1 a.bin b.bin
# SHA1(a.bin)= a34473cf767c6108a5751a20971f1fd6ba97690a
# SHA1(b.bin)= 4283dd2d70af1ad3c2d5fdc917330bf502035658
```

SHA-1 también los distingue: esta colisión está fabricada específicamente contra MD5. Las colisiones de SHA-1 existen (SHAttered), pero hay que construirlas contra SHA-1.

Para ver el ataque de sufijo común en acción, añade los mismos bytes al final de ambos archivos y vuelve a calcular: los MD5 siguen coincidiendo.

```
printf 'cualquier sufijo' | tee -a a.bin >> b.bin
md5sum a.bin b.bin # de nuevo iguales (aunque distintos de 7905...)
```

Impacto real

- **Certificados X.509 falsos:** en 2008 se creó un certificado de CA fraudulento con el mismo MD5 que uno legítimo.
- **Flame (2012):** malware que falsificó una firma de Microsoft usando una colisión de MD5 para aparentar ser una actualización de Windows.
- **Integridad rota:** un `md5sum` que coincide ya no garantiza que un archivo no fue sustituido.

Mitigación

No uses MD5 ni SHA-1 para nada que dependa de resistencia a colisiones (firmas, certificados, verificación de descargas, deduplicación con implicaciones de seguridad). Usa **SHA-256** o superior. Para autenticar mensajes con clave, usa **HMAC** (ver el Lab 5 y el ejemplo «SHA-256 y HMAC»; el bloque *SHA2* de CipherFlow tiene además la explicación paso a paso en su panel **Proceso**). MD5 solo sobrevive como suma de verificación no adversarial (detectar corrupción accidental).